МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»

Навчально-науковий фізико-технічний інститут

Кафедра інформаційної безпеки

ЗВІТ

з дисципліни «Аналіз кіберінцидентів»

Практичне заняття № 7 — Хешування, чутливе до розташування (LSH)

Виконав: Вовкотруб Олександр Віталійович
Група: ФБ-61мн

Перевірив: Наконечна Юлія Володимирівна

Київ — 2026

Зміст

1 Мета роботи **2**

2 Постановка задачі **2**

2.1 Реалізація 2

3 Хід виконання **3**

3.1 Компроміс між повнотою та кількістю кандидатів 3

3.2 Класифікація 3

4 Результати **4**

5 Аналіз **5**

5.1 LSH виявився повільнішим за точний пошук, і це головний результат 5

5.2 Порівняння з раніше реалізованими алгоритмами 5

5.3 Переваги та недоліки LSH 6

6 Висновки **6**

7 Джерела **6**

1 Мета роботи

Реалізувати метод хешування, чутливого до розташування (LSH), застосувати його до задачі класифікації зображень і порівняти результат з раніше реалізованими алгоритмами.

2 Постановка задачі

Використано датасет MNIST: 10 000 зображень рукописних цифр 28×28 у відтінках сірого, 784 ознаки-пікселі, 10 збалансованих класів.

Звичайний пошук найближчих сусідів (практичне заняття № 5) вимагає порівняти запит з кожним об’єктом вибірки. LSH дозволяє цього уникнути: він будує хеш-функції, для яких близькі об’єкти з високою ймовірністю потрапляють в один кошик, а далекі в різні. Тоді достатньо переглянути лише вміст кошика запиту. Це протилежність звичайному хешуванню, де схожі ключі навмисно розносяться.

2.1 Реалізація

У `scikit-learn` LSH відсутній (клас `LSHForest` вилучено з бібліотеки), тому алгоритм реалізовано власноруч.

Використано схему випадкових гіперплощин (SimHash) для косинусної подібності. Кожна гіперплощина задається випадковим вектором r, а біт підпису визначає, з якого боку від неї лежить точка:

$$h_r(x) = \begin{cases} 1, & r \cdot x > 0 \\ 0, & r \cdot x \le 0 \end{cases}$$

Ймовірність збігу біта для двох векторів дорівнює $1 - \theta/\pi$, де θ кут між ними: що менший кут, то вища ймовірність збігу. b гіперплощин дають b-бітний підпис; об’єкти з однаковим підписом складають кошик. Щоб зменшити ризик «розминутися» через невдалий набір гіперплощин, будується L незалежних таблиць, а кандидатами вважається об’єднання кошиків з усіх таблиць.



Рисунок 1: Приклади зображень

3 Хід виконання

3.1 Компроміс між повнотою та кількістю кандидатів

Параметри b і L керують основним компромісом методу. Вимірювався recall@5, тобто частка справжніх п’яти найближчих сусідів, яку знаходить LSH.

b (біт)	L (таблиць)	Recall@5	Кандидатів, %
8	1	0.2107	2.52
8	8	0.8540	22.62
8	16	0.9547	35.40
12	8	0.5587	4.13
16	8	0.3627	1.38
20	16	0.3353	0.61
24	16	0.2200	0.24

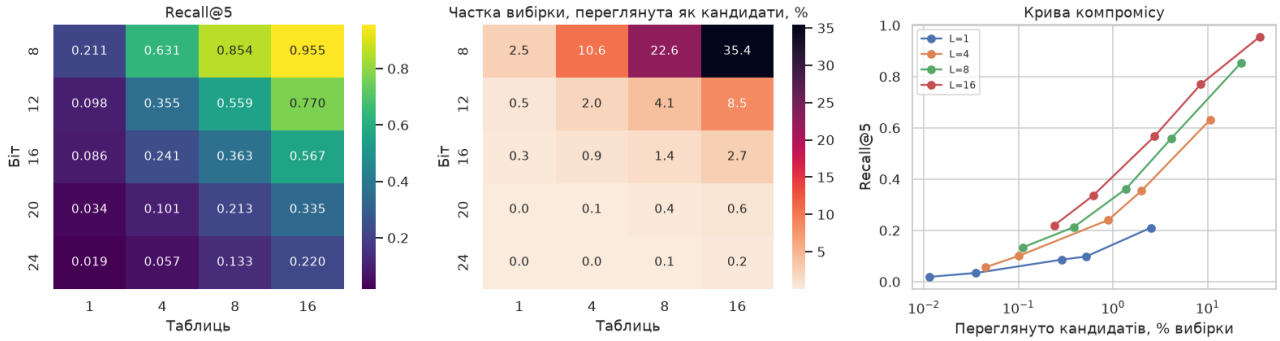


Рисунок 2: Компроміс повноти та швидкості

Закономірність однозначна: більше біт означає дрібніші кошики, менше кандидатів і нижчу повноту. Довжина підпису 24 біти звужує перегляд до 0.24 % вибірки, але знаходить лише 22 % справжніх сусідів. Збільшення кількості таблиць компенсує втрату повноти ціною зростання кількості кандидатів.

3.2 Класифікація

b	L	Точність	Покриття	Кандидатів, %
8	4	0.9324	1.0000	10.73
12	8	0.9260	1.0000	4.16
16	16	0.9188	1.0000	2.82
20	16	0.8503	0.9968	0.64
24	16	0.7876	0.9624	0.25

«Покриття» це частка запитів, для яких кошик виявився непорожнім. За довгих підписів кошик запиту може не містити жодного об’єкта, і метод не дає відповіді взагалі: за $b = 24$ це стосується 3.8 % запитів. Точність рахувалась лише за тими запитами, де відповідь є, тож для довгих підписів вона ще й завищена.

Показово, що класифікація витриваліша за пошук сусідів: за $b = 16$, $L = 16$ recall@5 становить лише 0.5667, але точність класифікації 0.9188. Причина в тому, що для голосування не потрібні саме ті п’ять найближчих сусідів: достатньо п’яти достатньо схожих об’єктів того самого класу.

4 Результати

Модель	Точність	мс на запит	Навчання, с
SVM RBF (ПЗ № 3)	0.9604	2.070	8.5
Випадковий ліс (ПЗ № 2)	0.9560	0.014	0.54
Точний kNN (косинусна)	0.9536	0.122	0.003
Точний kNN (евклідова)	0.9444	0.114	0.003
LSH (b=8, L=4)	0.9324	4.031	—
Логістична регресія (ПЗ № 1)	0.8816	0.002	71.1
Наївний Баєс (ПЗ № 4)	0.8356	0.003	0.10
Дерево рішень (ПЗ № 2)	0.8092	0.0006	1.3

Час у цій таблиці це один прохід по всій тестовій вибірці (7 500 запитів). У порівнянні нижче використано стійкіший замір: мінімум із трьох повторів на 1 000 запитах, тому абсолютні значення там менші.

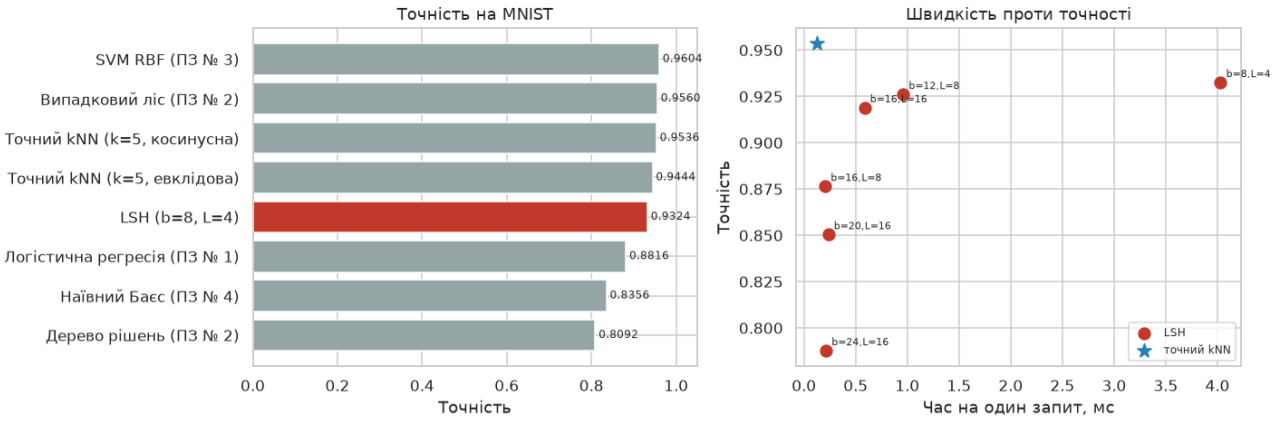


Рисунок 3: Порівняння методів

5 Аналіз

5.1 LSH виявився повільнішим за точний пошук, і це головний результат

Наївне порівняння дає несподіваний результат: LSH витрачає 2.45 мс на запит проти 0.12 мс у точного kNN зі `scikit-learn`, тобто приблизно у 20 разів повільніше, та ще й поступається в точності (0.9324 проти 0.9536).

Проте таке порівняння некоректне, бо змішує алгоритм і реалізацію. `scikit-learn` обчислює всю матрицю відстаней однією векторизованою операцією BLAS на скомпільованому коді; наша реалізація обходить запити циклом на Python з операціями над множинами. Щоб відокремити одне від одного, LSH порівняно з повним перебором, написаним у тому самому стилі:

Порівняння	Час на запит	Результат
LSH проти векторизованого `scikit-learn`	2.45 проти 0.12 мс	LSH приблизно у 20× повільніший
LSH проти перебору в тому самому стилі	2.45 проти 4.30 мс	LSH у 1.75× швидший
Алгоритмічна економія	10.6 % вибірки	у 9.4× менше обчислень відстані

Отже, алгоритмічний виграш реальний: переглядається 10.6 % вибірки замість 100 %. Але цей виграш у 9.4 рази перетворюється лише на 1.4–2.3-кратне прискорення (за кількома повторами заміру), бо частину заощадженого часу з'їдає власна робота LSH: обчислення проєкцій, пакування бітів і об'єднання кошиків. Показово, що стійким є саме алгоритмічний показник: частка переглянутих кандидатів відтворюється точно, тоді як співвідношення часу коливається між запусками залежно від завантаження машини.

Практичний висновок: на 7 500 об'єктів LSH не окупається. Метод є асимптотичним, його перевага виявляється на мільйонах записів, коли повна матриця відстаней не вміщується в пам'ять або обчислюється надто довго. На малих і середніх даних добре оптимізований повний перебір виграє за рахунок векторизації, попри гіршу асимптотику.

5.2 Порівняння з раніше реалізованими алгоритмами

LSH (0.9324) поступився точному kNN (0.9536) на 2.1 пункта, і це плата за наближеність. Обидва методи, своєю чергою, поступилися SVM (0.9604) і випадковому лісу (0.9560).

Цікаво, що на MNIST порядок методів змінився порівняно з попередніми роботами. Дерево рішень, яке домінувало на табличних даних UGRansome (0.9943), тут дає найгірший результат, 0.8092. Причина в природі ознак: піксель сам по собі майже нічого не означає, важлива їхня сукупна конфігурація. Дерево ж розбиває простір по одному пікселю за раз і змушене будувати величезні конструкції, щоб описати форму цифри. Методи, що працюють з усім вектором одразу (SVM, kNN, LSH), тут природніші.

Наївний Баєс (0.8356), який виграв на текстових даних у практичному занятті № 4, на зображеннях також відстає: піксельні інтенсивності не є незалежними, а сусідні пікселі сильно корельовані, тож наївне припущення порушується значно грубіше, ніж для слів.

5.3 Переваги та недоліки LSH

Переваги. Сублінійна складність пошуку; кількість переглянутих кандидатів регулюється параметрами; підписи компактні (кілька біт замість 784 чисел), що дає економію пам'яті; метод природно розподіляється між машинами, бо кошики незалежні.

Недоліки. Результат наближений, 2.1 пункта точності втрачено; два параметри (b і L) потребують підбору під конкретні дані; можливі порожні кошики, коли відповіді немає взагалі (3.8 % запитів за $b = 24$); виграш проявляється лише на великих обсягах, а на малих метод програє векторизованому перебору.

6 Висновки

1. Алгоритм LSH на випадкових гіперплощинах реалізовано власноруч (у `scikit-learn` він відсутній) і застосовано до класифікації зображень MNIST; найкраща конфігурація ($b = 8$, $L = 4$) дала точність 0.9324.
2. Компроміс «повнота проти швидкості» виміряно кількісно: довжина підпису 24 біти звужує перегляд до 0.24 % вибірки, але знаходить лише 22 % справжніх сусідів; за 8 біт і 16 таблиць recall@5 сягає 0.9547 ціною перегляду 35 %.
3. Класифікація витриваліша за пошук сусідів: за recall@5 = 0.5667 точність класифікації становить 0.9188, бо для голосування потрібні не саме найближчі сусіди, а достатньо схожі об'єкти того самого класу.
4. LSH програє точному kNN за часом на цих даних (приблизно у 20 разів), і це пояснюється масштабом та реалізацією, а не хибністю методу: у порівнянні «як з як» LSH швидший у 1.4–2.3 раза, а алгоритмічна економія становить 9.4 раза. LSH є асимптотичним методом і окупається на значно більших обсягах.
5. Порядок методів на зображеннях відрізняється від табличних даних: дерево рішень падає з 0.9943 (UGRansome) до 0.8092, бо окремий піксель неінформативний; виграють методи, що працюють з вектором цілком: SVM (0.9604) та kNN (0.9536).

7 Джерела

1. MNIST Dataset. URL: https://www.kaggle.com/datasets/wasiqaliyasir/digital-mnist-dataset
2. Charikar M. Similarity Estimation Techniques from Rounding Algorithms // STOC. 2002. P. 380–388.
3. Indyk P., Motwani R. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality // STOC. 1998. P. 604–613.

4. Leskovec J., Rajaraman A., Ullman J. Mining of Massive Datasets. Ch. 3: Finding Similar Items. Cambridge University Press, 2020.